

11 OBSERVER PATTERN (BEHAVIOURAL, OBJECT)

Purpose: When 1 SUBJECT changes, lots of observers need to know

- 1 Subject, many Observers; Subject notifies Observers on state changes

Use case: social media, live updates, e-commerce

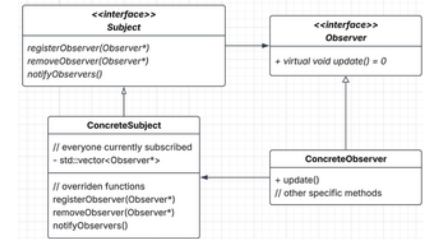
- subject holds vector of observers*

```
class Observer {
public:
    virtual ~Observer() = default;
    virtual void registerObserver(Observer* o) = 0;
    virtual void removeObserver(Observer* o) = 0;
    virtual void notifyObservers() = 0;
};

// Observer interface
class Observer {
public:
    virtual ~Observer() = default;
    virtual void update() = 0;
};

// Concrete Subject
class ConcreteSubject : public Subject {
    std::vector<Observer*> observers;
    int state = 0;
public:
    void registerObserver(Observer* o) override {
        observers.push_back(o);
    }
    void removeObserver(Observer* o) override {
        auto it = std::find(observers.begin(), observers.end(), o);
        if (it != observers.end())
            observers.erase(it);
    }
    void notifyObservers() override {
        for (Observer* o : observers)
            o->update();
    }
    void setState(int newState) {
        state = newState;
        notifyObservers();
    }
};

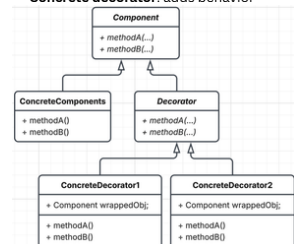
// Concrete Observer
class ConcreteObserver : public Observer {
    ConcreteSubject* subject;
public:
    explicit ConcreteObserver(ConcreteSubject* subject) :
        subject(subject) {}
    ~ConcreteObserver() {
        subject->removeObserver(this);
    }
    void update() override {
        int state = subject->getState(); // Pull data
        // React to the new state
    }
};
```



12 DECORATOR PATTERN (STRUCTURAL, OBJECT)

Purpose: attaches responsibilities to object w/o modifying original class

- Gives extension @ runtime, alt to subclassing
- **Concrete component:** original object being decorated
- **Concrete decorator:** adds behavior



```
// Component interface
class Component {
public:
    virtual ~Component() = default;
    virtual int operation() const = 0;
};

// Concrete interface
class ConcreteComponent : public Component {
public:
    int operation() const override (return 10);
};

// Base decorator: IS-A Component and HAS-A Component
class Decorator : public Component {
protected:
    Component* component;
public:
    ~Decorator() {}
    explicit Decorator(Component* component) :
        component(component) {}
};

// Concrete Decorator
class ConcreteDecorator : public Decorator {
public:
    explicit ConcreteDecorator(Component* component) :
        Decorator(component) {}
    int operation() const override {
        return component->operation() + 5;
    }
};
```

15 TEMPLATE METHOD PATTERN (BEHAVIOURAL, CLASS)

Purpose: workflow is fixed, indiv. step implementation varies

hook: default/empty method; subclass may override.

```
class AbstractProcess {
public:
    virtual ~AbstractProcess() = default;
    // Template method: fixed algorithm structure.
    virtual void run() final { //FINAL!!
        commonStart();
        requiredStep1();
        requiredStep2();
        if (useOptionalStep()) {
            optionalStep1();
        }
        commonEnd();
    }
protected:
    //start/end is virtual if depend on subclass
    virtual void commonStart() { shared setup; }
    virtual void commonEnd() { shared cleanup; }
    // Primitive operations: subclass MUST implement.
    virtual void requiredStep1() = 0;
    virtual void requiredStep2() = 0;
    // Hook: subclass MAY override.
    virtual bool useOptionalStep() const { return true; }
    virtual void optionalStep1() {}
};

class ConcreteProcess : public AbstractProcess {
protected:
    void requiredStep1() override { ... }
    void requiredStep2() override { ... }
    bool useOptionalStep() const override {
        return false; // optional
    }
};

// Client:
ConcreteProcess p;
p.run();
```

13.1 SIMPLE FACTORY PATTERN (CREATIONAL, OBJECT) (INHERITANCE)

Purpose: moves object creation into its own obj

Product: Things being created

Creator: Creates products

DP: Depend on abstraction not concrete class

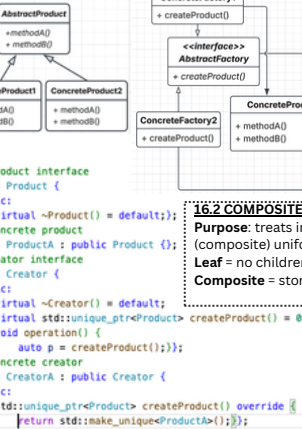
- no var should hold ref to a concrete class
- no class should derive from concrete class
- no method should override implement met. of base class

13.2 ABSTRACT FACTORY PATTERN (CREATIONAL, CLASS)

COMPOSITION:

Purpose: families of related objects

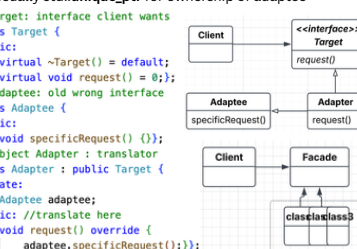
gets created



14.1 ADAPTER PATTERN (STRUCTURAL, OBJECT)

Purpose: Allows classes w incompatible interfaces to work tg

- **Target:** The interface the client wants
 - **Adaptee:** The existing incompatible class - wraps Adapter
 - **Adapter:** The translator - implement Target
 - **Client:** Existing code - uses Target interface
- (Object adapter: has-a, Class adapter: is-a)
- Usually std::unique_ptr for ownership of adaptee



14.2 Facade Pattern (structural, object)

Purpose: solves complex subsystems by providing 1 simple interface

Facade: HAS-A subsystem classes

Client: Talks only to facade

Members may be:

- **objects** (simplest)
- **unique_ptr** (exclusive ownership)
- **shared_ptr** (shared ownership)
- usually **shared_ptr**

```
// subsystem
class A { public: void f() {} };
class B { public: void g() {} };
class C { public: void h() {} };

// Facade HAS-A subsystem classes
class Facade {
private:
    A a; B b; C c;
public:
    void operation() {
        a.f(); b.g(); c.h();
    }
};

// View: display only
class StudentView {
public:
    void display(const StudentModel &m) const {
        std::cout << m.getName() << '\n';
    }
};

// Controller: handles actions + coordinates Model/View
class StudentController : public Observer {
    StudentModel &m; // non-owning
    StudentView &view; // non-owning
public:
    StudentController(StudentModel &m, StudentView &v) :
        m(m), view(v) {}
    ~StudentController() override {
        m.rem(this);
    }
    void changeName(std::string n) {
        m.setName(std::move(n));
    }
    void changeScore(double s) {
        m.setScore(s);
    }
    void update() override {
        // Model changed - refresh View
        view.display(m);
    }
};
```

```
// Observer pattern support
class Observer {
public:
    virtual ~Observer() = default;
    virtual void update() = 0;
};

// Subject (observer pattern)
class Subject {
    std::vector<Observer*> obs; // non-owning
public:
    void reg(Observer* o) { obs.push_back(o); }
    void remove(Observer* o) {
        auto it = std::find(obs.begin(), obs.end(), o);
        if (it != obs.end()) obs.erase(it);
    }
    void notify() {
        for (auto o : obs) o->update();
    }
};

// ConcreteElementA
class ConcreteElementA {
public:
    ~ConcreteElementA() = default;
    void accept(Visitor* v) { v->visit(*this); }
};

// ConcreteElementB
class ConcreteElementB {
public:
    ~ConcreteElementB() = default;
    void accept(Visitor* v) { v->visit(*this); }
};

// ConcreteVisitor1
class ConcreteVisitor1 {
public:
    ~ConcreteVisitor1() = default;
    void accept(ConcreteElementA &a) { a.accept(this); }
    void accept(ConcreteElementB &b) { b.accept(this); }
};

// ConcreteVisitor2
class ConcreteVisitor2 {
public:
    ~ConcreteVisitor2() = default;
    void accept(ConcreteElementA &a) { a.accept(this); }
    void accept(ConcreteElementB &b) { b.accept(this); }
};

// ObjectStructure
class ObjectStructure {
    vector<unique_ptr<ConcreteElement>> elements;
public:
    void accept(Visitor* v) {
        for (auto e : elements) e->accept(v);
    }
};
```

Concrete Visitor - One complete operation

class PrintVisitor : public Visitor

public:

void visit(const ElementA &a) override {

std::cout << "A: " << a.getValue() << '\n';

void visit(const ElementB &b) override {

std::cout << "B: " << b.getText() << '\n';

16.1 ITERATOR PATTERN

Purpose: Sequential access to collection w/o exposing implementation

Iterator Interface: same interface for all clients, **Concrete Iterator:** knows internal implementation

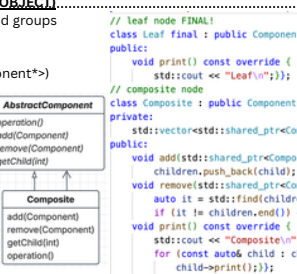
Concrete Aggregate: acc collection

Cohesion: measure of how closely a class or a module supports a single purpose

- no need to define iterator, can usually use eg (std::vector::Menuitem)::const_iterator

```
// Abstract Aggregate: returns an iterator
class Menu {
public:
    using const_iterator =
        std::vector<Menuitem*>::const_iterator;
    virtual ~Menu() = default;
    virtual const_iterator begin() const = 0;
    virtual const_iterator end() const = 0;
};

// Concrete Aggregate
class DinnerMenu : public Menu {
private:
    std::vector<Menuitem*> items;
public:
    void addItem(Menuitem* item) {
        items.push_back(std::move(item));
    }
    const_iterator begin() const override {
        return items.begin();
    }
    const_iterator end() const override {
        return items.end();
    }
};
```



17.1 STATE PATTERN (BEHAVIOURAL, OBJECT)

Purpose: Allows object to behave differently depending on current state

State interface: declares state-dependent actions

- 1 ConcreteState (often implement transitions) class per state

Context: owns shared data + reusable state objects

```
// Concrete States
class StateA : public State {
    Context &ctx; // back-reference
public:
    explicit StateA(Context &ctx) : ctx(ctx) {}
    virtual void action() override {
        // State A behaviour... trans
        ctx.setState(std::move(this));
    }
};

// State B behaviour... trans
class StateB : public State {
    Context &ctx;
public:
    explicit StateB(Context &ctx) : ctx(ctx) {}
    virtual void action() override {
        // State B behaviour... trans
        ctx.setState(std::move(this));
    }
};

// Context
class Context {
public:
    virtual ~Context() = default;
    virtual void action() = 0;
    virtual void request() = 0;
    virtual void set(State* s) { state = s; }
    virtual void get() const { return state; }
};

// Context implementation
class ConcreteContext : public Context {
public:
    ~ConcreteContext() = default;
    void request() override {
        // delegate
        state->action();
    }
    void set(State* s) { state = s; }
    void get() const { return state; }
};
```

18 MVC PATTERN (MODEL VIEW CONTROL)

Purpose: Split program into 3 parts with each their own responsibility

- **Want** to separate app logic from the UI
 - **Model:** Business logic (uses observer pattern)
 - **View:** only for display (uses composite pattern)
 - **Controller:** Receives user input & tells model what to do (strategy)
- Observer: Model changes -> Updates all views
- Composite: Views are often trees
- Strategy: Controller is the view's strategy * (opt)
- Adapter (optional): Adapting a new model to an existing view & controller

```
// Model: data + business logic
class StudentModel {
public:
    ~StudentModel() = default;
    std::string name;
    double score;
public:
    StudentModel(std::string n, double s) :
        name(std::move(n)), score(s) {}
    std::string getName() const { return name; }
    double getScore() const { return score; }
    void setName(std::string n) {
        name = std::move(n);
        notify(); // tell observers Model changed
    }
    void setScore(double s) {
        score = s;
        notify();
    }
};
```

```
// Forward declarations of concrete elements
class ElementA;
class ElementB;
// Visitor: one overload per concrete element
class Visitor {
public:
    virtual ~Visitor() = default;
    virtual void visit(const ElementA &a) = 0;
    virtual void visit(const ElementB &b) = 0;
};

// Element interface
class Element {
public:
    virtual ~Element() = default;
    virtual void accept(Visitor* v) const = 0;
};

// Concrete elements
class ElementA : public Element {
    int value;
public:
    explicit ElementA(int value) :
        value(value) {}
    int getValue() const {
        return value;
    }
    void accept(Visitor* v) const override {
        v->visit(*this);
    }
};

class ElementB : public Element {
    std::string text;
public:
    explicit ElementB(std::string text) :
        text(std::move(text)) {}
    std::string getText() const {
        return text;
    }
    void accept(Visitor* v) const override {
        v->visit(*this);
    }
};
```

Concrete Visitor - One complete operation

class PrintVisitor : public Visitor

public:

void visit(const ElementA &a) override {

std::cout << "A: " << a.getValue() << '\n';

void visit(const ElementB &b) override {

std::cout << "B: " << b.getText() << '\n';

```
class Component {
public:
    virtual ~Component() = default;
    // define ops that may not be supported
    virtual void add(std::shared_ptr<Component> c) {
        throw std::runtime_error("Unsupported");
    }
    virtual void print() const = 0;
protected:
    Component() = default;
};
```

16.3 COMPOSITE-ITERATOR PATTERN* (BEHAV, OBJ)

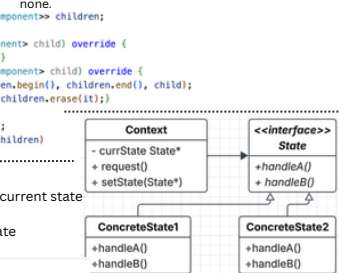
Traverses entire tree without exposing tree structure.

- Usually stores Component* in stack.

- Preorder: return node before children.

- Push children in reverse to preserve left-to-right order.

- If node is Composite, schedule children; Leaf adds none.



17.2 STRATEGY PATTERN (BEHAV, OBJ)

Purpose: Encapsulates interchangeable algos

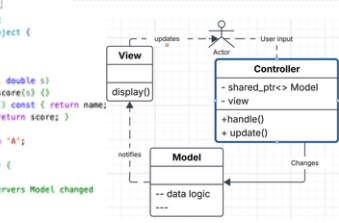
- context HAS-A strat
- **strategy** interface defines algorithm
- concrete strategy implement algorithm
- client can choose / replace algo @ runtime

```
// Strategy interface
class Strategy {
public:
    virtual ~Strategy() = default;
    virtual void execute() = 0;
};

// Concrete strategy
class StrategyA : public Strategy {
public:
    void execute() override { /* algorithm A */ }
};

class StrategyB : public Strategy {
public:
    void execute() override { /* algorithm B */ }
};

// Context HAS-A Strategy
class Context {
    std::unique_ptr<Strategy> strat;
public:
    ~Context() = default;
    void setStrategy(std::unique_ptr<Strategy> s) {
        strat = std::move(s);
    }
    void execute() { strat->execute(); }
};
```



```
// View: display only
class StudentView {
public:
    void display(const StudentModel &m) const {
        std::cout << m.getName() << '\n';
    }
};

// Controller: handles actions + coordinates Model/View
class StudentController : public Observer {
    StudentModel &m; // non-owning
    StudentView &view; // non-owning
public:
    StudentController(StudentModel &m, StudentView &v) :
        m(m), view(v) {}
    ~StudentController() override {
        m.rem(this);
    }
    void changeName(std::string n) {
        m.setName(std::move(n));
    }
    void changeScore(double s) {
        m.setScore(s);
    }
    void update() override {
        // Model changed - refresh View
        view.display(m);
    }
};
```

Concrete Visitor - One complete operation

class PrintVisitor : public Visitor

public:

void visit(const ElementA &a) override {

std::cout << "A: " << a.getValue() << '\n';

void visit(const ElementB &b) override {

std::cout << "B: " << b.getText() << '\n';

